PLN

Ciência de Dados para Negócios

FATEC Votorantim

Prof. Luis Claudio

M.Sc. Eng. Elétrica(PUC-Rio-06)

Eng. de Computação (ITA-99)

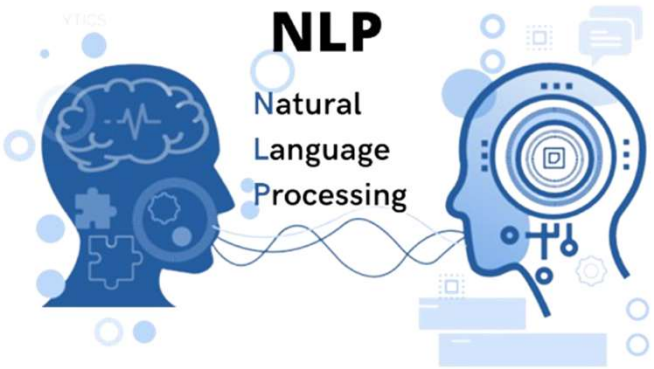
I –
O QUE É PLN

Prof. Luis Claudio
M.Sc. Eng. Elétrica(PUC-Rio-06)
Eng. de Computação (ITA-99)

O QUE É PLN

O Processamento de Linguagem Natural (PLN) é o campo que estuda como "extrair informações não triviais e de alto valor" a partir da linguagem humana. O desafio é transformar textos oriundos da linguagem natural em algo **computacionalmente tratável**.

Por definição, a **linguagem natural** não possui estrutura formal explícita, ou seja, é não estruturada. Portanto, precisa ser transformada em **representações estruturadas** antes que técnicas analíticas possam ser aplicadas de forma sistemática.



PROPRIEDADE DA LINGUAGEM NATURAL

Ambiguidade (Polissemia)

Ambiguidade refere-se à possibilidade de uma **mesma expressão linguística admitir mais de uma interpretação legítima**. Ela pode ser **sintática**, quando diferentes estruturas gramaticais são possíveis, ou **semântica**, quando uma palavra possui múltiplos sentidos, exigindo mecanismos de desambiguação em sistemas de PLN.

- Já a palavra "**banco**" pode significar instituição financeira ou assento. Aqui não há mudança estrutural, mas sim múltiplos sentidos associados ao mesmo termo, caracterizando ambiguidade semântica (ou lexical).
- Na frase "**Vi o homem com o telescópio**", há duas leituras estruturais plausíveis: ou eu utilizei o telescópio para ver o homem, ou o homem observado estava com o telescópio. É um caso de ambiguidade sintática.

PROPRIEDADE DA LINGUAGEM NATURAL

Variabilidade (Sinonímia)

A variabilidade diz respeito ao fato de que a **mesma ideia** pode ser expressa de muitas **maneiras diferentes**. Ao contrário da ambiguidade, aqui não existem vários sentidos possíveis. Existe um único sentido, mas com múltiplas formas de expressão. Isso obriga sistemas de PLN a reconhecer que textos diferentes podem estar dizendo essencialmente a mesma coisa.

- Por exemplo, "**rápido**", "**veloz**" e "**ágil**" transmitem ideias próximas. Pior ainda ocorre com frases inteiras: "**o sistema não funcionou**", "**o sistema caiu**", "**o sistema travou**" ou "**deu pau**". Todas descrevem o mesmo evento com escolhas linguísticas distintas. Essa multiplicidade de formas é natural na comunicação humana, mas aumenta demasiadamente a complexidade do processamento automático.

PROPRIEDADE DA LINGUAGEM NATURAL

Dependência de Contexto

A frase não tem vários sentidos disputando entre si, como acontece na ambiguidade. O problema é outro: a **informação linguística sozinha não basta**. Parte do significado vem da situação, do histórico da conversa ou do conhecimento compartilhado entre os interlocutores. Ou seja, o contexto não serve para escolher um sentido, mas para **completar** aquilo que não foi dito explicitamente.

- O significado de uma palavra depende das palavras ou da situação ao redor. Por exemplo: "banco" em "sentei no banco" versus "fui ao banco". A dependência de contexto surge justamente porque existem fenômenos como polissemia e sinonímia
- Ou ainda, na frase "**Era um lugar absurdamente frio**", também não há múltiplos sentidos disputando entre si. No entanto, sem contexto, não é possível determinar se a expressão é um exagero retórico (uma sala com ar-condicionado forte) ou a um ambiente extremo, como o Polo Norte.

PRINCIPAIS TAREFAS EM PLN

Podemos listar diversas **tarefas** (ou **problemas**) em PLN, isto é, o tipo de processamento que desejamos executar sobre texto ou fala.

- Na literatura, também é comum encontrar a expressão ***problemas*** *de PLN*, usada no sentido de formulação técnica de uma tarefa (por exemplo, um problema de classificação ou de tradução).

PRINCIPAIS TAREFAS EM PLN

Classificação Supervisionada

Classificação de textos consiste em **atribuir a um texto um rótulo dentre um conjunto finito e predefinido de classes**. Trata-se de um problema supervisionado no qual o modelo aprende, a partir de exemplos rotulados, uma função que mapeia representações vetoriais do texto para categorias específicas. A tarefa pode envolver classificação binária, multiclasse ou multilabel, dependendo do número e da natureza dos rótulos.

- Exemplo: dado um conjunto de ***e-mails*** rotulados como "***spam***" e "***não spam***", o modelo aprende padrões linguísticos e estatísticos e passa a prever o rótulo de novos e-*mails*. Isso não é "agrupar por similaridade" (o que seria um aprendizado não supervisionado), pois a saída é um rótulo predefinido, não *clusters* descobertos. A classificação pode ser **multiclasse** inclusive (até dezenas...).

PRINCIPAIS TAREFAS EM PLN

Análise de Sentimento

É um **tipo de classificação de texto** que consiste em **inferir a polaridade ou atitude em um texto** (por exemplo, positivo, negativo ou neutro). Na prática, é formulada como um problema de **classificação supervisionada**, no qual o modelo aprende a mapear representações textuais para classes de polaridade a partir de exemplos previamente rotulados.

Existem abordagens baseadas em léxicos, nas quais se utiliza um dicionário que associa palavras a escores numéricos de polaridade ou intensidade emocional (*SentiLex*, português, *VADER*, inglês etc.).

- Exemplo: na frase "O atendimento **foi excelente**", o sistema deve identificar um sentimento positivo; já em "O serviço **não foi dos melhores**", ele deve identificar um sentimento negativo.

PRINCIPAIS TAREFAS EM PLN

Reconhecimento de Entidades Nomeadas (NER)

Reconhecimento de entidades nomeadas, ou **NER** (*Named Entity Recognition*), consiste em identificar, em uma sequência textual, trechos **que correspondem a entidades do mundo real** e classificá-los em categorias predefinidas. Trata-se de um problema de rotulação sequencial, no qual cada *token* recebe um rótulo segundo um esquema de anotação (como PER, para pessoa, ORG, para a organização, LOC, para local, entre outros).

- Exemplo: no texto "Maria trabalha na Petrobras em São Paulo", o sistema identifica "Maria" como PER, "Petrobras" como ORG e "São Paulo" como LOC, estruturando essa informação em campos que podem alimentar uma base de conhecimento.

PRINCIPAIS TAREFAS EM PLN

Identificação de Tópicos

Identificação de tópicos visa descobrir automaticamente **temas recorrentes em uma coleção de documentos**, sem rótulos prévios. É, em geral, uma **abordagem tipicamente não supervisionada**, pois identifica padrões de coocorrência de termos e relações de significado ao longo dos textos analisados. O resultado são grupos temáticos caracterizados por conjuntos de palavras representativas e por documentos associados a esses temas.

- Exemplo: em milhares de comentários sobre eleições, o modelo pode identificar temas recorrentes como “corrupção”, “economia” e “segurança pública”, sem que essas categorias tenham sido previamente definidas. A partir disso, pode-se aplicar análise de sentimento (nesse caso, uma tarefa supervisionada) para avaliar a polaridade das opiniões em cada tema.

PRINCIPAIS TAREFAS EM PLN

Sumarização (saída = sequência textual)

Sumarização automática consiste em **gerar uma representação condensada de um texto** preservando seu conteúdo informacional central e sua coerência discursiva. Pode ser **extrativa**, quando seleciona e reorganiza trechos do próprio texto original com base em critérios de relevância, ou **abstrativa**, quando produz novas sentenças que sintetizam o conteúdo por meio de geração de linguagem. Em termos de formulação, pode ser supervisionada (treinada com pares texto–resumo) ou baseada em métodos não supervisionados de ranqueamento e relevância semântica.

- Exemplo: dado um artigo científico extenso, o sistema deve gerar um resumo que apresente problema, método e principais resultados, mantendo fidelidade factual e eliminando redundâncias.

PRINCIPAIS TAREFAS EM PLN

Geração de Texto (saída = sequência textual)

Consiste em **produzir novos conteúdos textuais a partir de uma instrução, pergunta ou contexto inicial**, modelando a geração como previsão sequencial de tokens condicionada ao histórico fornecido. O objetivo é gerar sequências coerentes, gramaticalmente corretas e semanticamente adequadas à solicitação. O avanço recente dessa área está associado aos **LLMs** (*Large Language Models*), baseados em arquiteturas ***Transformer*** e treinados em larga escala com aprendizado auto-supervisionado.

- Exemplo: ao receber a instrução "Explique, em dois parágrafos, o que é PLN", o sistema deve gerar uma resposta clara e consistente, respeitando o formato pedido.

PRINCIPAIS TAREFAS EM PLN

Tradução Automática (saída = sequência textual)

A tradução automática consiste em **converter um texto de um idioma para outro,** preservando seu sentido e produzindo uma **saída natural na língua de destino**. É uma das tarefas mais abrangentes do PLN porque integra simultaneamente desambiguação lexical, dependência de contexto, modelagem sintática e adequação pragmática. O sistema precisa interpretar corretamente o significado na língua de origem e reconstruí-lo de forma equivalente na língua de chegada, evitando traduções literais inadequadas.

- Exemplo: traduzir "This exam was a piece of cake" exige que o sistema reconheça que a expressão é idiomática e significa "Essa prova foi muito fácil", e não "Essa prova foi um pedaço de bolo". Aqui o desafio envolve identificar linguagem figurada e produzir uma equivalência natural no idioma de destino.

PRINCIPAIS TAREFAS EM PLN

Geração de Legendas (saída = sequência textual)

Consiste em **produzir, a partir do áudio de um vídeo, uma transcrição sincronizada em texto, geralmente com marcação temporal**. Em sua forma mais comum, envolve reconhecimento automático de fala para converter áudio em texto. É tipicamente formulada como um problema supervisionado, treinado com grandes volumes de pares áudio–transcrição. A qualidade é frequentemente avaliada por métricas como taxa de erro de palavras, além de critérios de sincronização temporal.

- Exemplo: em um vídeo educacional, o sistema gera automaticamente legendas alinhadas ao tempo de fala do professor, permitindo acessibilidade para pessoas com deficiência auditiva ou para usuários que assistem sem áudio.

PRINCIPAIS TAREFAS EM PLN

Perguntas e Respostas (Q&A)

Sistemas de perguntas e respostas recebem uma **pergunta em linguagem natural e produzem uma resposta,** normalmente com base em um contexto (documentos, trechos ou base de conhecimento). Em geral é supervisionado, treinado com pares pergunta–resposta ou com dados anotados de evidência.

- Exemplo: dado um trecho que contém "Marie Curie ganhou dois prêmios Nobel, em Física e em Química", a pergunta "Quantos prêmios Nobel Marie Curie ganhou?" deve ser respondida com "Dois", idealmente indicando como evidência o trecho "ganhou dois prêmios Nobel".

PRINCIPAIS TAREFAS EM PLN

Reconhecimento de Fala (multimodal)

Converte sinal de áudio da fala humana em texto, enfrentando variações como ruído, sotaque, ritmo, entonação e coarticulação. Em geral, é **treinado de forma supervisionada**, utilizando **grandes volumes de áudio alinhados a transcrições humanas**. A avaliação costuma empregar métricas como taxa de erro de palavras (*Word Error Rate - WER*).

- Exemplo: transcrever uma frase falada rapidamente, distinguindo “para” de “pra” e tratando hesitações, pausas e repetições de modo consistente com o objetivo do sistema.

PRINCIPAIS TAREFAS EM PLN

Descrição de Imagens (multimodal)

Consiste em **gerar uma descrição textual em linguagem natural a partir do conteúdo visual de uma imagem**. Trata-se de uma tarefa multimodal, pois combina visão computacional (extração de características visuais) com PLN. Em geral, é formulada como um problema supervisionado, utilizando pares imagem–descrição para treinar modelos capazes de mapear representações visuais para sequências textuais coerentes. Arquiteturas modernas frequentemente utilizam modelos baseados em **redes neurais profundas** com mecanismos de atenção ou ***transformers*** multimodais.

- Exemplo: dada uma imagem contendo "uma mulher atravessando a rua com um guarda-chuva em um dia chuvoso", o sistema deve produzir uma descrição equivalente em texto. É especialmente relevante para tecnologias assistivas voltadas a pessoas com deficiência visual.

CONCEITOS

Em Processamento de Linguagem Natural, trabalha-se tipicamente com **conjuntos textuais definidos para análise**, denominados *corpus*. O termo é preferido por indicar um conjunto selecionado com critérios metodológicos, diferentemente da expressão genérica "dados".

CORPUS

Conjunto de textos selecionado intencionalmente, com recorte analítico claro e finalidade definida.

CORPORA

Plural acadêmico de *corpus*, utilizado quando se comparam diferentes conjuntos textuais.

II – PROCESSO DE ANÁLISE DE DADOS

Prof. Luis Claudio
M.Sc. Eng. Elétrica(PUC-Rio-06)
Eng. de Computação (ITA-99)

PROCESSO DE ANÁLISE DE DADOS

Independentemente da técnica, projetos de análise de dados costumam seguir um conjunto recorrente de etapas que vão do entendimento do problema à geração de *insights* acionáveis.

Esse processo foi formalizado sob o termo **KDD** (*Knowledge Discovery in Databases*) em 1996 para descrever a **descoberta de conhecimento** por meio de **etapas** como seleção, pré-processamento, transformação, **mineração** e interpretação.

Essa mesma lógica foi posteriormente estruturada em metodologias operacionais como o **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*), amplamente adotado para organizar e conduzir **projetos de mineração de dados** em ambientes organizacionais. As etapas eram *Business Understanding*, *Data Understanding*, *Data Preparation*, *Modeling*, *Evaluation*, *Deployment*.

PROCESSO DE ANÁLISE DE DADOS

Vamos adotar um processo analítico genérico, organizado em **seis macroetapas** proposto em processos de treinamento de certificação oficial da Google*:

- *Entendimento*
- *Preparação*
- *Pré-processamento*
- *Análise*
- *Compartilhamento*
- *Ação*

ENTENDIMENTO

Nesta etapa definimos qual fenômeno linguístico queremos investigar e qual tarefa de PLN será formulada. Diferentemente de análises tabulares, aqui a pergunta precisa ser traduzida em termos de linguagem: que tipo de texto será analisado, qual comportamento textual se deseja identificar e qual saída o sistema deverá produzir.

Isso envolve explicitar se o objetivo é, por exemplo, **classificar documentos, medir sentimento, responder perguntas** etc.

A **pergunta analítica passa a ser formulada como uma tarefa sobre a linguagem**, com definição clara da unidade observacional, do tipo de inferência desejada e do critério de sucesso da análise.

PREPARAÇÃO

A etapa de preparação antecede qualquer tratamento computacional do texto. Seu foco é definir **qual material linguístico será utilizado e em que condições ele será analisado**.

Aqui decidimos quais **fontes textuais** fazem sentido para o problema proposto, qual recorte será considerado, qual unidade textual será adotada (por exemplo, documentos, sentenças ou interações) e se o conjunto disponível é adequado para responder à pergunta analítica.

Isso inclui atividades como identificar repositórios ou sistemas de origem, selecionar coleções documentais, verificar cobertura e representatividade do *corpus*, avaliar volume de dados, analisar condições de acesso e **critérios de inclusão ou exclusão de textos**.

PRÉ-PROCESSAMENTO

Pré-processar dados textuais significa preparar o corpus para análise linguística, assegurando que o conjunto de documentos seja representativo, coerente e tecnicamente utilizável. O objetivo não é apenas integridade formal, mas garantir que o material reflita adequadamente o fenômeno linguístico estudado, evitando distorções introduzidas por **ruídos de coleta ou problemas de origem**.

Incluem-se aqui atividades como remoção de documentos duplicados ou quase duplicados, eliminação de ruído estrutural típico de textos extraídos da web (**menus, anúncios, marcações HTML**), correção de problemas de codificação de caracteres, **filtragem de textos truncados** ou fora do escopo analítico, verificação e padronização de metadados e organização do *corpus* em formato adequado para processamento computacional.

ANÁLISE

Agora analisamos o *corpus* já processado para responder à pergunta analítica por meio de técnicas de PLN.

O foco deixa de ser a organização do material e passa a ser sua representação analítica, isto é, a **transformação da linguagem em estruturas** que permitam identificar padrões, relações e comportamentos linguísticos relevantes.

Entram nesta fase as operações próprias do *pipeline* de PLN, como segmentação textual, normalização linguística, construção de **representações vetoriais** e geração de atributos derivados do texto, além da aplicação de métodos estatísticos ou de aprendizado de máquina quando necessários.

COMPARTILHAMENTO

Compartilhar é comunicar os resultados da análise textual de forma clara e adequada ao público que utilizará o sistema ou interpretará seus achados.

Em projetos de PLN, isso pode envolver explicar como o texto foi representado, quais tarefas foram executadas (classificação, extração, sumarização, geração etc.), quais métricas foram utilizadas para avaliação e quais limitações o sistema apresenta.

A forma de apresentação é decisiva. É necessário contextualizar o *corpus* utilizado, explicitar possíveis vieses linguísticos, delimitar escopo de aplicação e esclarecer limites do modelo diante de ambiguidade, variações de contexto e mudanças no uso da linguagem.

AÇÃO

Agir, em projetos de PLN, consiste em **aplicar o sistema desenvolvido em um contexto real de uso**. Isso pode significar integrar um classificador a um fluxo de atendimento, incorporar um mecanismo de extração de entidades a um sistema jurídico, utilizar um modelo de sentimento para monitoramento de opinião ou implantar um sistema de geração ou resposta automática em ambiente produtivo.

Assim como em outros projetos analíticos, a etapa de ação inclui acompanhamento contínuo. Sistemas de PLN são sensíveis a mudanças linguísticas, novos termos, gírias, variações culturais e alterações no domínio textual. Portanto, esta etapa envolve monitorar desempenho, revisar *corpus* quando necessário, atualizar modelos e avaliar impacto prático das decisões automatizadas. O ciclo permanece iterativo: novas falhas observadas ou novos usos da linguagem podem demandar ajustes no modelo e reinício do processo analítico.

III - PIPELINE PLN
ENGENHARIA DE ATRIBUTOS

Prof. Luis Claudio
M.Sc. Eng. Elétrica(PUC-Rio-06)
Eng. de Computação (ITA-99)

PIPELINE EM PLN

Chamamos de *pipeline* o encadeamento de etapas técnicas executadas dentro da macroetapa ***Analisar***, nas quais os textos são transformados em representações computacionais e utilizados para especificar, **treinar, validar e avaliar modelos de PLN**.

É esse fluxo que detalharemos a seguir, a saber:

- * Engenharia de atributos
- Especificação do modelo
- Treinamento
- Validação
- Teste

* Utilizamos a expressão "Engenharia de atributos" como um **conceito guarda-chuva** para designar qualquer transformação orientada ao modelo, isto é, decisões que moldam os dados para torná-los adequados a alguma tarefa específica de PLN.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Engenharia de atributos (*feature engineering*) é o processo de criar, transformar, selecionar ou agregar variáveis a partir dos dados brutos com o **objetivo de melhorar a capacidade do modelo**.

As etapas específicas variam conforme o tipo de dado, mas o princípio é o mesmo: transformar informação bruta em representação útil ao modelo. Em **PLN**, isso envolve:

- **Segmentação** (definir unidades linguísticas de análise)
- **Normalização textual** (reduzir variações formais não informativas)
- **Extração de informação** (estruturar elementos linguísticos relevantes)
- **Representação vetorial do texto** (converter texto em espaço numérico modelável)

- Em **PLN**, o dado bruto é **texto**, que não é diretamente modelável. O texto não nasce estruturado como variáveis prontas para modelagem. Portanto, a *feature engineering* envolve estruturar a própria representação do dado antes da geração de atributos derivados.

- Em PLN ***feature construction*** e ***feature selection*** são frequentemente efeitos das etapas anteriores, enquanto em dados tabulares elas são etapas explícitas e separadas

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

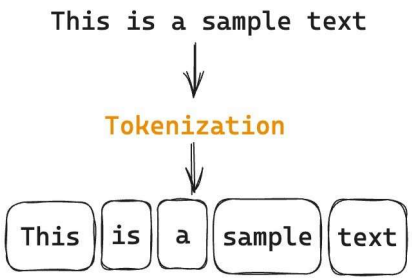
Tokenização

Tokenização é o processo de "**segmentar o texto em unidades básicas** chamadas ***tokens***", que servirão como entrada para o modelo. Ela define qual será a **unidade de análise linguística** e, portanto, influencia o tipo de padrão que o modelo poderá aprender.

- Por exemplo, na **análise de sentimento**, costuma-se utilizar tokenização por **palavras**, pois o objetivo é capturar termos como "excelente", "péssimo" ou "horrível". Em tarefas como descrição de imagens e geração de legendas, modelos modernos baseados em ***Transformers*** utilizam tokenização por **subpalavras**.

Obs.: não há uma regra fixa associada à tarefa. A decisão técnica depende do modelo adotado e da estratégia analítica.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS



Tokenização por Palavras

Consiste em segmentar o texto em unidades correspondentes a **palavras**, delimitadas por espaços ou sinais de pontuação segundo regras linguísticas ou heurísticas.

- Exemplo: "O sistema falhou novamente."
  Tokens: ["O", "sistema", "falhou", "novamente"]

É a abordagem típica do PLN clássico, sendo amplamente utilizada em representações como: Bag-of-Words, TF-IDF.

Seu principal benefício é a **interpretabilidade**, pois o modelo opera diretamente sobre **palavras explícitas do vocabulário**.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Tokenização por Subpalavras

Consiste em segmentar palavras em **unidades menores que uma palavra** com base na sua **frequência estatística** dentro no *corpus*.

- Exemplo: “infelizmente”
  Tokens possíveis: [“in”, “feliz”, “mente”]

Essa abordagem é dominante em modelos modernos baseados em ***Transformers***, como: BERT, RoBERTa e GPT.

Seu principal benefício é reduzir o problema de palavras fora do vocabulário, controlar o tamanho do vocabulário e permitir melhor generalização para palavras raras ou compostas. É adequada quando se trabalha com grandes modelos neurais e vocabulário aberto (caso moderno dos LLMs.).

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Tokenização por palavra (BoW, TF-IDF)
Corpus grande → facilmente 500 mil, 1 milhão ou mais palavras distintas. Cada uma vira um token diferente.

- Resultado:
  Vocabulário enorme.
  Muitas palavras raras.
  Muitas "palavras nunca vistas".

```
feliz
infeliz
infelizmente
felicidade
felicíssimo
```

Tokenização por subpalavra (BERT, RoBERTa e GPT)
O vocabulário é fixado previamente (BERT, 30k tokens, GPT 50K tokens) e aprendido estatisticamente a partir do corpus.

```
feliz
##mente
in
```

```
in + feliz + ##mente
```

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Tokenização por Caracteres

Consiste em segmentar o texto em **caracteres individuais**.

- Exemplo: “falha”
  Tokens: [“f”, “a”, “l”, “h”, “a”]

É utilizada em abordagens específicas que buscam robustez a erros ortográficos, variações morfológicas ou quando se deseja modelar padrões muito finos da linguagem. É menos frequente como estratégia principal em arquiteturas modernas de grande escala.

As sequências tornam-se maiores e o custo computacional aumenta.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Normalização

Consiste em aplicar "transformações ao texto com o objetivo de **reduzir variações que não agregam distinção relevante**" para a tarefa de modelagem. O objetivo não é alterar o conteúdo semântico, mas tornar a representação textual mais consistente, reduzindo redundâncias e variações morfológicas que poderiam aumentar artificialmente a dimensionalidade do espaço de atributos.

- Em termos práticos, a normalização pode envolver **remoção de palavras** pouco informativas, **redução de palavras** às suas formas básicas e **padronização de formas** linguísticas.
- Em PLN, algumas transformações associadas à *Preparação* também podem ser tratadas como orientadas ao modelo, tais como: *lowercasing*, remoção de **acentos**, tratamento de **pontuação**, correção **ortográfica** e normalização de **números** (datas, valores etc.). Isso porque, em PLN, a aplicação depende da tarefa e da **sensibilidade do modelo a essas variações** ("Varejo", "varejo" ou "VAREJO", "Premium", "Premiun" ou "prêmium").

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Normalização – Remoção de Stopwords

Consiste em eliminar **palavras muito frequentes** que, em determinadas tarefas, contribuem pouco para diferenciar documentos. Geralmente incluem artigos, preposições e conjunções.

- Exemplo : "O sistema não está funcionando corretamente."
  Após remoção de stopwords: ["sistema", "funcionando", "corretamente"]

É comum em representações clássicas como *Bag-of-Words* e TF-IDF, especialmente em classificação de textos e identificação de tópicos.

Seu objetivo é reduzir **ruído estatístico e dimensionalidade**, mantendo termos com maior poder informativo.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Normalização – Stemming

Stemming consiste em **reduzir palavras ao seu radical** por meio de regras heurísticas que removem sufixos e terminações ("truncam").

- Exemplo: "caminhando", "caminhou", "caminhada"
  Após stemming: "**caminh**"

O radical obtido pode não corresponder a uma palavra válida no idioma, mas permite agrupar variações morfológicas sob uma mesma base. É utilizado principalmente em abordagens clássicas para reduzir **variação lexical e dimensionalidade**.

Seu objetivo é tratar flexões como **manifestações de um mesmo núcleo semântico**.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Normalização – Lematização

Lematização consiste em reduzir palavras à sua forma canônica (**lema**), considerando a **morfologia flexional.**

- Exemplo: "falando", "falou", "falei", "falaram"
  Após lematização: "**falar**"

Diferentemente do *stemming*, a lematização **preserva validade linguística** e **realiza análise morfológica.**

gênero (bonito / bonita)

número (carro / carros)

tempo verbal (amou / amando / amarão)

Atenção: a lematização **não** trata **derivação, prefixação** ou **<u>sufixação</u>**.

derivação (feliz ≠ felicidade ≠ infelizmente)

prefixação (feliz ≠ infeliz)

sufixação produtiva

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔍 **Extração de Informação**

Consiste em **identificar e estruturar elementos linguísticos relevantes** presentes no texto, transformando conteúdo não estruturado em representações organizadas e interpretáveis.

O objetivo é explicitar padrões que não estão formalmente delimitados no texto bruto, como **categorias gramaticais, entidades,** relações ou **estruturas sintáticas.**

Diferentemente da normalização, que **reduz** variações formais, a extração de informação **adiciona estrutura analítica ao texto,** tornando explícitos componentes que podem ser utilizados como **atributos para modelos mais complexos.**

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Extração de Informação – POS Tagging

POS (*Part-of-Speech*) consiste em atribuir a cada palavra **sua classe gramatical**, como substantivo, verbo, adjetivo ou advérbio.

- Exemplo: "O sistema falhou novamente."
  Anotação possível: O (DET), sistema (NOUN), falhou (VERB), novamente (ADV)

Essa técnica adiciona estrutura gramatical explícita ao texto, permitindo analisar padrões sintáticos e gerar atributos derivados, como proporção de substantivos ou frequência de verbos.

Seu objetivo é **estruturar a dimensão morfossintática** do texto.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔍 **Extração de Informação – Named Entity Recognition (NER)**

Consiste em **identificar e classificar entidades nomeadas no texto**, como pessoas, organizações, locais, datas ou valores monetários.

- Exemplo: "Maria trabalha na Petrobras desde 2020."
  NER: Maria → Pessoa, Petrobras → Organização, 2020 → Data

NER pode ser o objetivo final da tarefa (extração automática de entidades) ou uma etapa intermediária para geração de atributos, como contagem de entidades ou identificação de atores relevantes.

Seu objetivo é **estruturar elementos explícitos** presentes no discurso.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

- **Extração de Informação – Parsing Sintático**

Parsing sintático consiste em identificar as relações gramaticais entre as palavras de uma sentença, construindo uma estrutura que representa dependências sintáticas.

- Exemplo: "O sistema apresentou falhas."
  apresentou → verbo principal,
  sistema → sujeito de apresentou,
  falhas → objeto de apresentou

Tecnicamente, o parsing sintático é a implementação computacional da análise sintática tradicional da linguística. Ela explicita como os termos se relacionam na estrutura da frase, permitindo **análises mais profundas de significado e relações.**

Seu objetivo é **estruturar a organização sintática da sentença.**

CANCELADO

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 **Representação Vetorial**

Converter texto em vetores ou matrizes para uso por modelos. Como os modelos operam sobre **vetores e matrizes,** o texto precisa ser transformado **preservando padrões linguísticos relevantes.**

Em síntese, representar vetorialmente é transformar linguagem em **geometria computacional.**

A representação define **o que será preservado** no texto:

- **Frequência lexical (contagem de termos)**
- **Coocorrência estatística (padrões de contexto)**
- **Similaridade semântica (relações semânticas)**

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Representação Vetorial

Por Frequência Lexical

(**BoW** e **TF-IDF**)

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 **Representação Vetorial – Frequência Lexical**

Esparsas (clássicas): vetores de **alta dimensionalidade** baseados em contagem/peso de termos (BoW, TF-IDF). O significado é representado pela presença e frequência explícita de termos no documento. Existe interpretabilidade lexical.

Documentos são caracterizados por distribuição lexical explícita, não por estrutura semântica latente. Similaridade é baseada em sobreposição de vocabulário, não em proximidade semântica (carro *versus* automóvel). Dois documentos são considerados próximos se **compartilham as mesmas palavras**.

Palavras são tratadas como unidades independentes cuja importância decorre de sua **frequência ou raridade no *corpus***.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 Representação Vetorial – One-Hot Encoding (básico)

Cada ***token*** é representado por um vetor binário do tamanho do vocabulário. A representação é **esparsa** (quase todas as posições são 0, com um único 1 por *token*) e de alta dimensionalidade (cada vetor tem dimensão |V|).

- Se o vocabulário é composto por: **modelo, análise, dados, sistema, algoritmo,** temos a seguinte representação ***one-hot***:

  modelo → [1, 0, 0, 0, 0]
  análise → [0, 1, 0, 0, 0]
  dados → [0, 0, 1, 0, 0]
  sistema → [0, 0, 0, 1, 0]
  algoritmo → [0, 0, 0, 0, 1]

  Imagine o tamanho dos vetores para um **|V| = 10000 tokens**?

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 Representação Vetorial – BoW (clássico)

O ***Bag-of-Words*** representa o **documento inteiro** como um único vetor de dimensão |V| (tamanho do vocabulário). Cada posição indica a **frequência** de um ***token*** no documento (dessa forma ignorando a ordem das palavras). A representação é esparsa e de alta dimensionalidade.

- Vocabulário: **modelo, análise, dados, sistema, algoritmo**
  Texto: "um modelo de análise de dados é um modelo inteligente"
  Representação no **BoW**: [2, 1, 1, 0, 0]
- Ou seja, 2 ocorrências de "modelo", 1 de "análise", 1 de "dados", nenhuma de "sistema" ou "algoritmo". A posição do elemento no vetor indica o *token* no vocabulário e a quantidade de vezes em que ele aparece no documento.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

O que vira um vetor no BoW?

No BoW, **cada unidade de análise vira um vetor.**

Essa unidade pode ser:

- um documento inteiro (mais comum)
- um parágrafo
- uma frase
- um título
- um comentário
- um e-mail
- uma notícia

Depende da tarefa.

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 Representação Vetorial – TF-IDF (clássico)

O *Term Frequency – Inverse Document Frequency* também representa cada documento como um vetor de dimensão |V|, ponderando a frequência de cada token pela sua **raridade no *corpus***. Enquanto o BoW considera só contagem bruta, o TF-IDF reduz o peso de termos muito frequentes no conjunto de documentos e aumenta o peso de termos mais informativos. A representação continua sendo esparsa e de alta dimensionalidade.

- Formalmente: **TF-IDF(t, d) = TF(t, d) × IDF(t)**
- onde:
  TF(t, d) = frequência do token *t* no documento *d*
  IDF(t) = log (N / df(t))

  N = # total de documentos
  df(t) = # de documentos que contêm o token *t*
- Se um termo aparece em quase todos os documentos, seu peso diminui.
- Se aparece em poucos documentos, seu peso aumenta.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

TF-IDF(t, d) = TF(t, d) × IDF(t), ou

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

Onde:

- $tf_{i,j}$ = número de ocorrências de *i* em *j*
- df_i = número de documentos contendo *i*
- *N* = número total de ~~ocorrências~~

Correções...

Word	TF A	TF B	IDF	TF*IDF A	TF*IDF B
The	2/7	2/7	log(2/2) = 0	0	0
Car	1/7	0	log(2/1) = 0.3	0.043	0
Truck	0	1/7	log(2/1) = 0.3	0	0.043
Is	1/7	1/7	log(2/2) = 0	0	0
Driven	1/7	1/7	log(2/2) = 0	0	0
On	1/7	1/7	log(2/2) = 0	0	0
~~The~~	~~1/7~~	~~1/7~~	~~log(2/2) = 0~~	~~0~~	~~0~~
Road	1/7	0	log(2/1) = 0.3	0.043	0
Highway	0	1/7	log(2/1) = 0.3	0	0.043

CANCELADO

A: *The car is driven on the road.* (O carro é conduzido na estrada)
B: *The truck is driven on the highway.* (O caminhão é conduzido na rodovia)

O valor calculado para TF-IDF das palavras comuns aos documentos foi igual a zero (**menos significativas**). Por outro lado, o valor para as palavras "car", "truck", "road" e "highway" é maior (**mais significativas**).

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

TF-IDF(t, d) = TF(t, d) × IDF(t), ou

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

onde:

tf_{ij} = frequência do termo i no documento j

N = número total de documentos no corpus

df_i = número de documentos que contêm o termo i

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	2/7	2/7	log(2/2) = 0	0	0
Car	1/7	0	log(2/1) = 0.3	0.043	0
Truck	0	1/7	log(2/1) = 0.3	0	0.043
Is	1/7	1/7	log(2/2) = 0	0	0
Driven	1/7	1/7	log(2/2) = 0	0	0
On	1/7	1/7	log(2/2) = 0	0	0
Road	1/7	0	log(2/1) = 0.3	0.043	0
Highway	0	1/7	log(2/1) = 0.3	0	0.043

A: *The car is driven on the road*. (O carro é conduzido na estrada)
B: *The truck is driven on the highway*. (O caminhão é conduzido na rodovia)

O valor calculado para TF-IDF das palavras comuns aos documentos foi igual a zero (**menos significativas**). Por outro lado, o valor para as palavras “car”, “truck”, “road” e “highway” é maior (**mais significativas**).

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

A: the car is driven on the road
B: the truck is driven on the highway
C: the man is walking on the floor

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

onde:

$tf_{i,j}$ = frequência do termo i no documento j

N = número total de documentos no corpus

df_i = número de documentos que contêm o termo i

O vetor TF-IDF destaca **as palavras que diferenciam os documentos,** ignorando os termos muito comuns.

Term	TF(A)	TF(B)	TF(C)	df	IDF	TF-IDF(A)	TF-IDF(B)	TF-IDF(C)
the	2/7	2/7	2/7	3	0	0	0	0
is	1/7	1/7	1/7	3	0	0	0	0
on	1/7	1/7	1/7	3	0	0	0	0
driven	1/7	1/7	0	2	0.176	0.025	0.025	0
car	1/7	0	0	1	0.477	0.068	0	0
road	1/7	0	0	1	0.477	0.068	0	0
truck	0	1/7	0	1	0.477	0	0.068	0
highway	0	1/7	0	1	0.477	0	0.068	0
man	0	0	1/7	1	0.477	0	0	0.068
walking	0	0	1/7	1	0.477	0	0	0.068
floor	0	0	1/7	1	0.477	0	0	0.068

(0.176 = log10(3/2); 0.477 = log10(3/1); 0.025 = (1/7)*0.176; 0.068 = (1/7)*0.477)

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

N-gramas (unigrama, bigrama e trigrama)

N-gramas são sequências de ***n* tokens** consecutivos extraídas de um texto. O objetivo é **capturar padrões locais de linguagem** que envolvem uma ou mais palavras. Dependendo do valor de *n*, a sequência representa diferentes níveis de contexto:

Unigram (*1-gram*) - Considera cada token individualmente.
Bigram (*2-gram*) - Considera pares de tokens consecutivos.
Trigram (*3-gram*) - Considera trios de tokens consecutivos.

- Exemplo: *eu gosto de ciência de dados*

Unigrama: *eu* | *gosto* | *de* | *ciência* | *de* | *dados*
Bigrama: *eu gosto* | *gosto de* | *de ciência* | *ciência de* | *de dados*
Trigrama: *eu gosto de* | *gosto de ciência* | *de ciência de* | *ciência de dados*

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

N-gramas em Bag-of-Words (BoW)

Cada palavra ou sequência de palavras se torna um atributo do vetor. Os **n-grams definem quais sequências serão contabilizadas.**

Texto:

`eu gosto de dados`

BoW com unigram

termo	frequência
eu	1
gosto	1
de	1
dados	1

BoW com bigram

termo	frequência
eu gosto	1
gosto de	1
de dados	1

BoW com Trigram

termo	frequência
eu gosto de	1
gosto de dados	1

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

N-gramas em TF-IDF

No TF-IDF, os **n-grams também definem atributos,** mas em vez de simples contagem o método aplica um **peso estatístico** baseado na **frequência no documento** e na **raridade no *corpus*.**

- **Exercício - Considere o seguinte *corpus* com três documentos:**
  D1: eu gosto de dados
  D2: eu gosto de ciência
  D3: dados e ciência

- Utilizando **bigramas (n = 2)**:
  Liste os **bigramas gerados em cada documento.**
  Calcule o **DF** de cada bigrama.
  Calcule o **IDF** de cada bigrama.
  Calcule o **TF-IDF para os bigramas do documento D1**.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

D1: eu gosto de dados
D2: eu gosto de ciência
D3: dados e ciência

1. Bigrams

D1 → eu gosto, gosto de, de dados
D2 → eu gosto, gosto de, de ciência
D3 → dados e, e ciência

2. TF por documento

bigrama	TF em D1	TF em D2	TF em D3
eu gosto	1	1	0
gosto de	1	1	0
de dados	1	0	0
de ciência	0	1	0
dados e	0	0	1
e ciência	0	0	1

3. DF de cada bigrama

bigrama	DF
eu gosto	2
gosto de	2
de dados	1
de ciência	1
dados e	1
e ciência	1

4. IDF de cada bigrama

Usando **IDF = log(3/DF)**

bigrama	IDF
eu gosto	log(3/2)
gosto de	log(3/2)
de dados	log(3)
de ciência	log(3)
dados e	log(3)
e ciência	log(3)

5. TF-IDF para D1

bigrama	TF em D1	IDF	TF-IDF
eu gosto	1	log(3/2) = **0,405**	0,405
gosto de	1	log(3/2) = **0,405**	0,405
de dados	1	log(3) = **1,099**	1,099

PIPELINE EM PLN
PROBLEMAS DA REPRESENTAÇÃO ESPARSA

1) Ambiguidade (Polissemia)

Problema: uma mesma palavra pode ter significados diferentes.

- Exemplo:
- "Ele sentou no banco."
- "Ele foi ao banco."

Em **BoW** e **TF-IDF,** "banco" é apenas um termo do vocabulário.

O modelo não entende sentidos (só conta **quantas vezes** cada palavra aparece).

PIPELINE EM PLN
PROBLEMAS DA REPRESENTAÇÃO ESPARSA

2) Variabilidade (Sinonímia)

Problema: palavras diferentes com mesmo significado.

- Exemplo:
  Doc A: "O carro é rápido."
  Doc B: "O automóvel é rápido."
  Doc C: " O veículo é veloz."

Tanto para **BoW** quanto para o **TF-IDF**: "carro" ≠ "automóvel" ≠ "veículo".

O modelo não entende sinônimos (só conta **quantas vezes** cada palavra aparece).

PIPELINE EM PLN
PROBLEMAS DA REPRESENTAÇÃO ESPARSA

3) Dependência de Contexto (Relações Semânticas Latentes)

Problema: textos podem falar do mesmo tema com outras palavras.

- Exemplo:
  Doc A: "Ele comprou ações na bolsa de valores."
  Doc B: "O mercado financeiro caiu devido à crise."
  Doc C: "O urso tomou conta do mercado."

Os dois falam sobre finanças. Mas não compartilham palavras iguais.

Em **BoW** e **TF-IDF**, a similaridade será baixa, pois o modelo só compara palavras idênticas. Ele não entende que "ações", "bolsa" e "mercado financeiro" pertencem ao mesmo campo de significado.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

Representação Vetorial

Por Coocorrência Estatística

(**PMI**)

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 **Representação Vetorial – Coocorrência Estatística**

Baseia-se em **quantas vezes palavras aparecem próximas umas das outras no *corpus*,** utilizando uma **janela de contexto**. Essas relações são organizadas em **matrizes,** nas quais cada palavra é representada pelos termos que costumam aparecer ao seu redor.

Técnicas como **PMI** exploram essas matrizes para identificar padrões estatísticos no uso das palavras.

A similaridade passa a depender **dos contextos compartilhados,** e não apenas da presença das mesmas palavras. Assim, palavras que aparecem em **contextos semelhantes** tendem a ser representadas de forma parecida, revelando **relações semânticas aproximadas derivadas da estrutura estatística da linguagem.**

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 **PMI – Pointwise Mutual Information**

PMI é uma medida usada em modelos de representação vetorial baseados em **coocorrência de palavras.** A técnica parte de uma **matriz palavra-contexto,** que registra quantas vezes uma palavra aparece **próxima de outra** dentro de um *corpus*.

A medida central é o **PMI (*Pointwise Mutual Information*),** que quantifica o quanto a coocorrência entre uma palavra e um contexto é **maior ou menor do que seria esperado pelo acaso**. Sua função é medir **quão fortemente uma palavra está associada a um contexto,** comparando a coocorrência observada com a coocorrência esperada ao acaso.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 PMI – Pointwise Mutual Information

Se duas palavras aparecem juntas com frequência maior que a esperada pela **probabilidade independente** de cada uma, o valor de PMI será positivo.

PMI (Pointwise Mutual Information)

Mede **quanto duas palavras aparecem juntas mais (ou menos) do que seria esperado ao acaso.**

$$PMI(w, c) = \log \frac{P(w, c)}{P(w)P(c)}$$

Interpretação:

- **PMI > 0** → associação positiva (aparecem juntas mais do que o esperado)
- **PMI = 0** → independência
- **PMI < 0** → aparecem juntas menos do que o esperado

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 **PPMI – Positive Pointwise Mutual Information**

O problema do PMI é que ele também pode assumir **valores negativos,** indicando que duas palavras aparecem juntas menos do que o esperado. Em representações vetoriais de linguagem, esses **valores negativos não são úteis.**

Por isso utiliza-se o **PPMI (Positive PMI),** que simplesmente substitui valores negativos por zero:

$$PPMI(w, c) = \max(PMI(w, c), 0)$$

Apenas **associações positivas entre palavras e contextos** são preservadas. O resultado é uma matriz de representação vetorial mais adequada para capturar **relações semânticas** entre palavras em modelos baseados em coocorrência.

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 **Probabilidade Conjunta e Independência de Eventos**

Considere o experimento de lançar **duas moedas consecutivamente.** Calcule:

- (a) a **probabilidade** de a **1ª moeda dar cara;**
- (b) a **probabilidade** de a 1ª **moeda dar cara e a 2ª também dar cara.**

Eventos independentes

$$P(A, B) = P(A) \times P(B)$$

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 Probabilidade Conjunta e Dependência de Eventos

Considere o experimento de **retirar duas cartas consecutivamente de um baralho, sem reposição**. Calcule:

- (a) a probabilidade de a **1ª carta ser um Ás;**
- (b) a probabilidade de a **1ª carta ser um Ás e a 2ª também ser um Ás.**

Eventos dependentes

$$P(A, B) = P(A) \times P(B|A)$$

Se **A influencia B**, então:

$$P(B|A) \neq P(B)$$

Ou seja, a ocorrência de **A altera a probabilidade de B.**

PIPELINE EM PLN
ENGENHARIA DE ATRIBUTOS

🔢 Probabilidade em um corpus

Em PLN, consideramos eventos como:

- (a) A: aparecer a **palavra w;**
- (b) A: aparecer a **palavra c;**

Probabilidade marginal

$$P(w) = \frac{\text{freq}(w)}{N}$$

$$P(c) = \frac{\text{freq}(c)}{N}$$

Probabilidade conjunta

$$P(w, c) = \frac{\text{cooc}(w, c)}{N}$$

onde:

- **cooc(w,c)** = número de vezes que as palavras aparecem juntas
- **N** = total de ocorrências consideradas

Ideia central: duas palavras são independentes quando a probabilidade de ocorrerem juntas é igual ao produto de suas probabilidades marginais, isto é, $P(w, c) = P(w) \times P(c)$

BOW

```
corpus = [
    "gato bebe agua",
    "cachorro bebe agua",
    "gato bebe leite",
    "cachorro bebe leite",
    "gato come peixe",
    "cachorro come peixe",
    "gato come carne",
    "cachorro come carne",
    "homem bebe agua",
    "homem come carne",
]
```

```
[('bebe', 5),
 ('come', 5),
 ('gato', 4),
 ('cachorro', 4),
 ('agua', 3),
 ('carne', 3),
 ('leite', 2),
 ('peixe', 2),
 ('homem', 2)]
```

bow

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
D1	1	1	0	0	0	1	0	0	0
D2	1	1	1	0	0	0	0	0	0
D3	0	1	0	0	0	1	0	1	0
D4	0	1	1	0	0	0	0	1	0
D5	0	0	0	0	1	1	0	0	1
D6	0	0	1	0	1	0	0	0	1
D7	0	0	0	1	1	1	0	0	0
D8	0	0	1	1	1	0	0	0	0
D9	1	1	0	0	0	0	1	0	0
D10	0	0	0	1	1	0	1	0	0

TF-IDF

```
corpus = [
    "gato bebe agua",
    "cachorro bebe agua",
    "gato bebe leite",
    "cachorro bebe leite",
    "gato come peixe",
    "cachorro come peixe",
    "gato come carne",
    "cachorro come carne",
    "homem bebe agua",
    "homem come carne",
]
```

```
[('bebe', 5),
 ('come', 5),
 ('gato', 4),
 ('cachorro', 4),
 ('agua', 3),
 ('carne', 3),
 ('leite', 2),
 ('peixe', 2),
 ('homem', 2)]
```

tfidf

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
D1	0.641771	0.512414	0.000000	0.000000	0.000000	0.570581	0.000000	0.000000	0.000000
D2	0.641771	0.512414	0.570581	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
D3	0.000000	0.482845	0.000000	0.000000	0.000000	0.537655	0.000000	0.691222	0.000000
D4	0.000000	0.482845	0.537655	0.000000	0.000000	0.000000	0.000000	0.691222	0.000000
D5	0.000000	0.000000	0.000000	0.000000	0.482845	0.537655	0.000000	0.000000	0.691222
D6	0.000000	0.000000	0.537655	0.000000	0.482845	0.000000	0.000000	0.000000	0.691222
D7	0.000000	0.000000	0.000000	0.641771	0.512414	0.570581	0.000000	0.000000	0.000000
D8	0.000000	0.000000	0.570581	0.641771	0.512414	0.000000	0.000000	0.000000	0.000000
D9	0.582818	0.465343	0.000000	0.000000	0.000000	0.000000	0.666168	0.000000	0.000000
D10	0.000000	0.000000	0.000000	0.582818	0.465343	0.000000	0.666168	0.000000	0.000000

PMI

```python
corpus = [
    "gato bebe agua",
    "cachorro bebe agua",
    "gato bebe leite",
    "cachorro bebe leite",
    "gato come peixe",
    "cachorro come peixe",
    "gato come carne",
    "cachorro come carne",
    "homem bebe agua",
    "homem come carne",
]
```

bigrams_list

```python
[('gato', 'bebe'),
 ('bebe', 'agua'),
 ('cachorro', 'bebe'),
 ('bebe', 'agua'),
 ('gato', 'bebe'),
 ('bebe', 'leite'),
 ('cachorro', 'bebe'),
 ('bebe', 'leite'),
 ('gato', 'come'),
 ('come', 'peixe'),
 ('cachorro', 'come'),
 ('come', 'peixe'),
 ('gato', 'come'),
 ('come', 'carne'),
 ('cachorro', 'come'),
 ('come', 'carne'),
 ('homem', 'bebe'),
 ('bebe', 'agua'),
 ('homem', 'come'),
 ('come', 'carne')]
```

bigram_freq

```python
Counter({('gato', 'bebe'): 2,
         ('bebe', 'agua'): 3,
         ('cachorro', 'bebe'): 2,
         ('bebe', 'leite'): 2,
         ('gato', 'come'): 2,
         ('come', 'peixe'): 2,
         ('cachorro', 'come'): 2,
         ('come', 'carne'): 3,
         ('homem', 'bebe'): 1,
         ('homem', 'come'): 1})
```

matrix de coocorrência a partir dos bigramas

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
agua	0	0	0	0	0	0	0	0	0
bebe	3	0	0	0	0	0	0	2	0
cachorro	0	2	0	0	2	0	0	0	0
carne	0	0	0	0	0	0	0	0	0
come	0	0	0	3	0	0	0	0	2
gato	0	2	0	0	2	0	0	0	0
homem	0	1	0	0	1	0	0	0	0
leite	0	0	0	0	0	0	0	0	0
peixe	0	0	0	0	0	0	0	0	0

PMI

```python
corpus = [
    "gato bebe agua",
    "cachorro bebe agua",
    "gato bebe leite",
    "cachorro bebe leite",
    "gato come peixe",
    "cachorro come peixe",
    "gato come carne",
    "cachorro come carne",
    "homem bebe agua",
    "homem come carne",
]
```

```python
bigrams_list
```

```
[('gato', 'bebe'),
 ('bebe', 'agua'),
 ('cachorro', 'bebe'),
 ('bebe', 'agua'),
 ('gato', 'bebe'),
 ('bebe', 'leite'),
 ('cachorro', 'bebe'),
 ('bebe', 'leite'),
 ('gato', 'come'),
 ('come', 'peixe'),
 ('cachorro', 'come'),
 ('come', 'peixe'),
 ('gato', 'come'),
 ('come', 'carne'),
 ('cachorro', 'come'),
 ('come', 'carne'),
 ('homem', 'bebe'),
 ('bebe', 'agua'),
 ('homem', 'come'),
 ('come', 'carne')]
```

```python
bigram_freq
```

```
Counter({('gato', 'bebe'): 2,
         ('bebe', 'agua'): 3,
         ('cachorro', 'bebe'): 2,
         ('bebe', 'leite'): 2,
         ('gato', 'come'): 2,
         ('come', 'peixe'): 2,
         ('cachorro', 'come'): 2,
         ('come', 'carne'): 3,
         ('homem', 'bebe'): 1,
         ('homem', 'come'): 1})
```

```python
# Calcular total de coocorrências.
N = matrix.values.sum()
N
```

```
np.int64(20)
```

```python
p_wc = matrix / N
p_wc
```

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
agua	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
bebe	0.15	0.00	0.0	0.00	0.00	0.0	0.0	0.1	0.0
cachorro	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
carne	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
come	0.00	0.00	0.0	0.15	0.00	0.0	0.0	0.0	0.1
gato	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
homem	0.00	0.05	0.0	0.00	0.05	0.0	0.0	0.0	0.0
leite	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
peixe	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0

PMI

```
corpus = [
    "gato bebe agua",
    "cachorro bebe agua",
    "gato bebe leite",
    "cachorro bebe leite",
    "gato come peixe",
    "cachorro come peixe",
    "gato come carne",
    "cachorro come carne",
    "homem bebe agua",
    "homem come carne",
]
```

bigrams_list

```
[('gato', 'bebe'),
 ('bebe', 'agua'),
 ('cachorro', 'bebe'),
 ('bebe', 'agua'),
 ('gato', 'bebe'),
 ('bebe', 'leite'),
 ('cachorro', 'bebe'),
 ('bebe', 'leite'),
 ('gato', 'come'),
 ('come', 'peixe'),
 ('cachorro', 'come'),
 ('come', 'peixe'),
 ('gato', 'come'),
 ('come', 'carne'),
 ('cachorro', 'come'),
 ('come', 'carne'),
 ('homem', 'bebe'),
 ('bebe', 'agua'),
 ('homem', 'come'),
 ('come', 'carne')]
```

p_w

	0
agua	0.00
bebe	0.25
cachorro	0.20
carne	0.00
come	0.25
gato	0.20
homem	0.10
leite	0.00
peixe	0.00

p_c

	0
agua	0.15
bebe	0.25
cachorro	0.00
carne	0.15
come	0.25
gato	0.00
homem	0.00
leite	0.10
peixe	0.10

bigram_freq

```
Counter({('gato', 'bebe'): 2,
         ('bebe', 'agua'): 3,
         ('cachorro', 'bebe'): 2,
         ('bebe', 'leite'): 2,
         ('gato', 'come'): 2,
         ('come', 'peixe'): 2,
         ('cachorro', 'come'): 2,
         ('come', 'carne'): 3,
         ('homem', 'bebe'): 1,
         ('homem', 'come'): 1})
```

```
# Calcular total de coocorrências.
N = matrix.values.sum()
N
```

```
np.int64(20)
```

```
p_wc = matrix / N
p_wc
```

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
agua	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
bebe	0.15	0.00	0.0	0.00	0.00	0.0	0.0	0.1	0.0
cachorro	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
carne	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
come	0.00	0.00	0.0	0.15	0.00	0.0	0.0	0.0	0.1
gato	0.00	0.10	0.0	0.00	0.10	0.0	0.0	0.0	0.0
homem	0.00	0.05	0.0	0.00	0.05	0.0	0.0	0.0	0.0
leite	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0
peixe	0.00	0.00	0.0	0.00	0.00	0.0	0.0	0.0	0.0

PMI

```
corpus = [
    "gato bebe agua",
    "cachorro bebe agua",
    "gato bebe leite",
    "cachorro bebe leite",
    "gato come peixe",
    "cachorro come peixe",
    "gato come carne",
    "cachorro come carne",
    "homem bebe agua",
    "homem come carne",
]
```

bigrams_list

```
[('gato', 'bebe'),
 ('bebe', 'agua'),
 ('cachorro', 'bebe'),
 ('bebe', 'agua'),
 ('gato', 'bebe'),
 ('bebe', 'leite'),
 ('cachorro', 'bebe'),
 ('bebe', 'leite'),
 ('gato', 'come'),
 ('come', 'peixe'),
 ('cachorro', 'come'),
 ('come', 'peixe'),
 ('gato', 'come'),
 ('come', 'carne'),
 ('cachorro', 'come'),
 ('come', 'carne'),
 ('homem', 'bebe'),
 ('bebe', 'agua'),
 ('homem', 'come'),
 ('come', 'carne')]
```

PMI é calculado por:

$$PMI(w,c) = \log(P(w,c) / (P(w) \times P(c)))$$

onde:

P(w,c) = probabilidade conjunta do par palavra–contexto

P(w) = probabilidade marginal da palavra

P(c) = probabilidade marginal do contexto

bigram_freq

```
Counter({('gato', 'bebe'): 2,
         ('bebe', 'agua'): 3,
         ('cachorro', 'bebe'): 2,
         ('bebe', 'leite'): 2,
         ('gato', 'come'): 2,
         ('come', 'peixe'): 2,
         ('cachorro', 'come'): 2,
         ('come', 'carne'): 3,
         ('homem', 'bebe'): 1,
         ('homem', 'come'): 1})
```

pmi

	agua	bebe	cachorro	carne	come	gato	homem	leite	peixe
agua	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
bebe	1.386294	0.000000	NaN	0.000000	0.000000	NaN	NaN	1.386294	0.000000
cachorro	0.000000	0.693147	NaN	0.000000	0.693147	NaN	NaN	0.000000	0.000000
carne	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
come	0.000000	0.000000	NaN	1.386294	0.000000	NaN	NaN	0.000000	1.386294
gato	0.000000	0.693147	NaN	0.000000	0.693147	NaN	NaN	0.000000	0.000000
homem	0.000000	0.693147	NaN	0.000000	0.693147	NaN	NaN	0.000000	0.000000
leite	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
peixe	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

IV - PIPELINE PLN
ESPECIFICAÇÃO DE MODELOS

Prof. Luis Claudio
M.Sc. Eng. Elétrica(PUC-Rio-06)
Eng. de Computação (ITA-99)

ESPECIFICAÇÃO DE MODELOS

Supervisionados Clássicos

- Regressão Linear *

* Os problemas em PLN são majoritariamente de classificação e não de previsão de valores contínuos.

- Regressão Logística
- Naive Bayes
- KNN (*K-Nearest Neighbors*)
- SVM (*Support Vector Machine*)
- Árvores / Floresta Aleatória (Random Forest)

Não Supervisionado Clássicos

- PCA (*Principal Component Analysis*)
- LDA (*Latent Dirichlet Allocation*)
- K-Means

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - PCA

PCA (*Principal Component Analysis*)

O PCA é uma técnica que transforma um conjunto de variáveis originais em um novo conjunto de variáveis chamadas **componentes principais.** Essas novas variáveis são construídas como **combinações lineares das variáveis originais** e têm como objetivo reorganizar os dados em um novo sistema de coordenadas.

Ao aplicar o PCA, cada observação deixa de ser representada pelas variáveis originais e passa a ser representada pelos valores nos componentes principais. O resultado é um novo espaço onde os dados são descritos por eixos que **capturam a maior variabilidade possível.**

Esses componentes são ordenados de acordo com a **quantidade de variância que explicam** (PC1 > PC2 > PC3 ...).

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - PCA

PCA (*Principal Component Analysis*)

Uma consequência direta dessa transformação é a possibilidade de **reduzir a dimensionalidade dos dados**. Como os primeiros componentes concentram a maior parte da variabilidade, é possível manter apenas alguns deles e descartar os demais, reduzindo o número de variáveis **sem perder grande parte da informação**.

Ao mesmo tempo, o PCA não apenas transforma os dados, mas aprende **uma regra de transformação a partir deles**. Essa regra define como as variáveis originais devem ser combinadas para gerar os componentes principais. Uma vez aprendida, essa transformação pode ser aplicada a novos dados.

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - PCA

Antes do PCA, tenho apenas as variáveis explicativas originais:

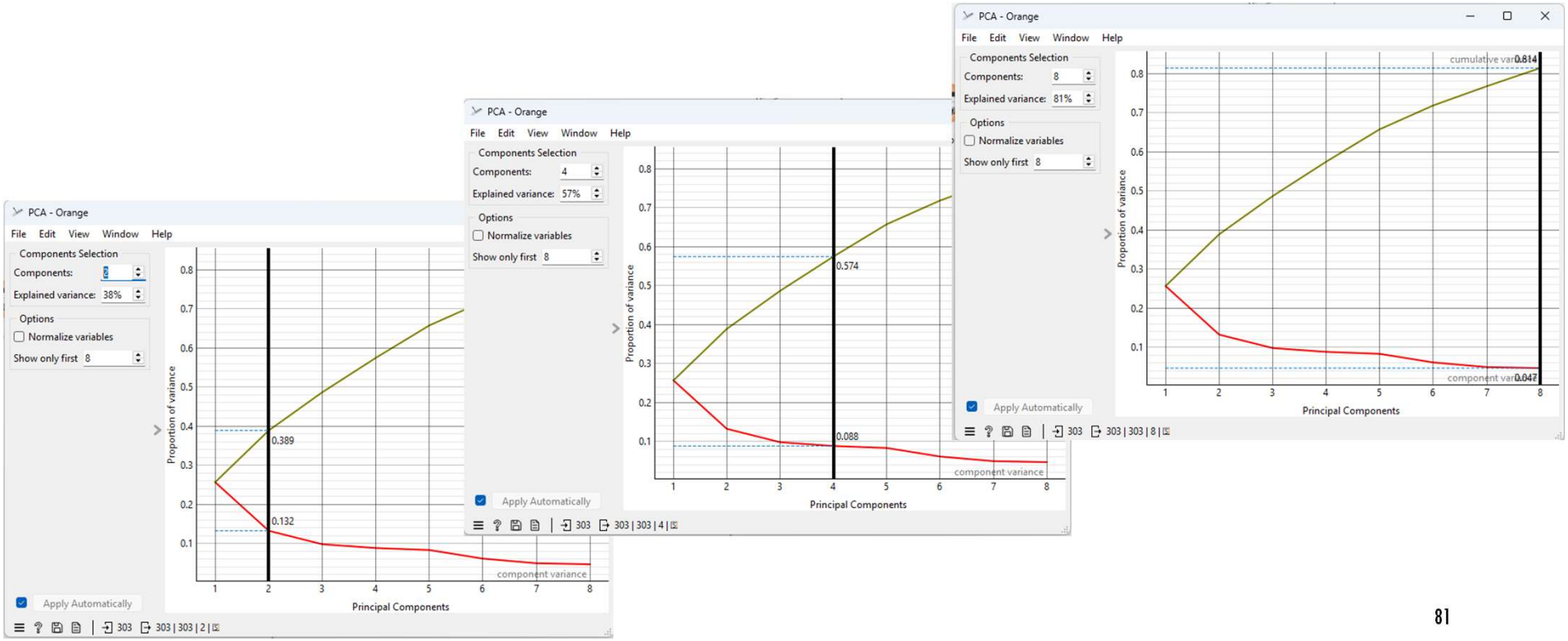
	iameter narrowin	age	gender	chest pain	rest SBP	cholesterol	ng blood sugar >	exerc ind ang	ajor vessels colore	thal	slope peak exc ST	ST by exercise	max HR	rest ECG
1	0	0.949	male	typical ang	0.758	-0.2649	1	0	-0.723	fixed defect	downsloping	1.087	0.017	left vent hypert...
2	1	1.392	male	asymptomatic	1.611	0.7604	0	1	2.504	normal	flat	0.397	-1.822	left vent hypert...
3	1	1.392	male	asymptomatic	-0.665	-0.3423	0	1	1.428	reversable defect	flat	1.346	-0.902	left vent hypert...
4	0	-1.933	male	non-anginal	-0.096	0.0640	0	0	-0.723	normal	downsloping	2.123	1.637	normal
5	0	-1.489	female	atypical ang	-0.096	-0.8259	0	0	-0.723	normal	upsloping	0.311	0.981	left vent hypert...
6	0	0.173	male	atypical ang	-0.665	-0.2069	0	0	-0.723	normal	upsloping	-0.207	1.243	normal
7	1	0.838	female	asymptomatic	0.473	0.4122	0	0	1.428	normal	downsloping	2.209	0.455	left vent hypert...
8	0	0.284	female	asymptomatic	-0.665	2.0759	0	1	-0.723	normal	upsloping	-0.379	0.586	normal
9	1	0.949	male	asymptomatic	-0.096	0.1414	0	0	0.353	reversable defect	flat	0.311	-0.114	left vent hypert...
10	1	-0.159	male	asymptomatic	0.473	-0.8453	1	1	-0.723	reversable defect	downsloping		0.236	left vent hypert...

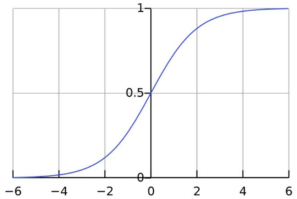
Após o PCA, terei os componentes principais (2, 3 etc...):

variance	iameter narrowin	PC1 0.256458	PC2 0.132157	age	gender	chest pain	rest SBP	cholesterol	ng blood sugar >	exerc ind ang	ajor vessels colore	thal	slope peak exc ST	ST by exercise	max HR	rest ECG
1	0	-0.83553	0.315858	0.949	male	typical ang	0.758	-0.2649	1	0	-0.723	fixed defect	downsloping	1.087	0.017	left vent hypert...
2	1	-3.52298	0.802846	1.392	male	asymptomatic	1.611	0.7604	0	1	2.504	normal	flat	0.397	-1.822	left vent hypert...
3	1	-2.65239	-1.11573	1.392	male	asymptomatic	-0.665	-0.3423	0	1	1.428	reversable defect	flat	1.346	-0.902	left vent hypert...
4	0	1.2955	-0.658518	-1.933	male	non-anginal	-0.096	0.0640	0	0	-0.723	normal	downsloping	2.123	1.637	normal
5	0	1.83815	-0.176243	-1.489	female	atypical ang	-0.096	-0.8259	0	0	-0.723	normal	upsloping	0.311	0.981	left vent hypert...
6	0	1.63133	0.176836	0.173	male	atypical ang	-0.665	-0.2069	0	0	-0.723	normal	upsloping	-0.207	1.243	normal
7	1	-1.84299	0.745071	0.838	female	asymptomatic	0.473	0.4122	0	0	1.428	normal	downsloping	2.209	0.455	left vent hypert...
8	0	0.546397	1.39261	0.284	female	asymptomatic	-0.665	2.0759	0	1	-0.723	normal	upsloping	-0.379	0.586	normal
9	1	-1.17028	-0.0226817	0.949	male	asymptomatic	-0.096	0.1414	0	0	0.353	reversable defect	flat	0.311	-0.114	left vent hypert...
10	1	-0.932747	-1.04341	-0.159	male	asymptomatic	0.473	-0.8453	1	1	-0.723	reversable defect	downsloping	1.777	0.236	left vent hypert...
11	0	0.25666	-0.621066	0.284	male	asymptomatic	0.473	-1.0581	0	0	-0.723	fixed defect	flat	-0.552	-0.070	normal
12	0	0.0551549	1.21279	0.173	female	atypical ang	0.473	0.9152	0	0	-0.723	normal	flat	0.225	0.149	left vent hypert...
13	1	-0.645763	-0.0695848	0.173	male	non-anginal	-0.096	0.1800	1	1	0.353	fixed defect	flat	-0.379	-0.333	left vent hypert...
14	0	2.02881	-0.0548133	-1.157	male	atypical ang	-0.665	0.3155	0	0	-0.723	reversable defect	upsloping	-0.897	1.024	normal

ESPECIFICAÇÃO DE MODELOS NÃO SUPERVISIONADO - PCA

A escolha de quantos PCs leva em consideração a quantidade de variância explicada. O principal benefício do PCA é reduzir a dimensionalidade. Logo, o número de componentes para a capturar uma quantidade grande de variância não pode ser tão grande quanto o número de variáveis explicativas original (como no caso abaixo).



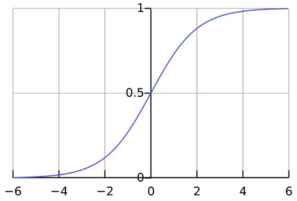


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

No aprendizado supervisionado, modelos são treinados a partir de dados rotulados (X → y), com o objetivo de **aprender a relação entre variáveis explicativas e a variável alvo** para realizar previsões.

Essa abordagem se divide em dois tipos de problema: **regressão**, quando o objetivo é prever **valores contínuos** (y contínuo), e **classificação**, quando o objetivo é prever **classes/categorias** (y discreto).

No contexto de classificação em PLN, a regressão logística é um dos modelos mais utilizados, devido à sua simplicidade (:o), eficiência e boa **interpretabilidade**.



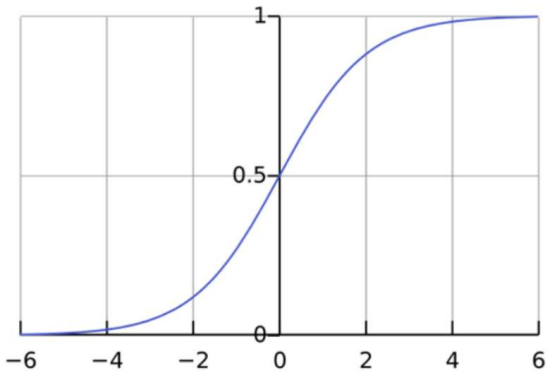
$P(y = 1 \mid x)$

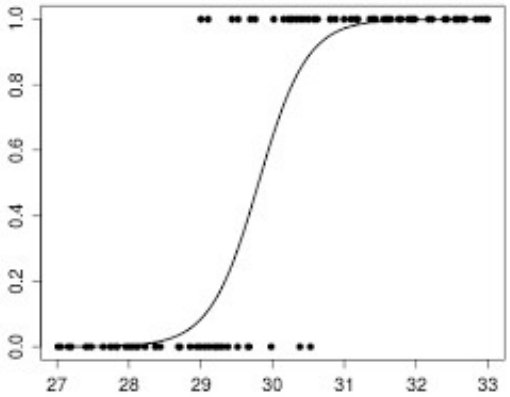
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

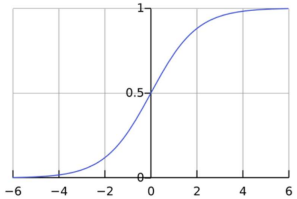
Em **PLN**, a classificação de texto é uma das tarefas mais comuns em PLN (sentimento, autoria, spam etc.). Nestes casos, a **regressão logística** é utilizada para estimar a **probabilidade de uma observação pertencer à classe 1**, dado um conjunto de atributos.

Para isso, o modelo transforma uma saída linear em uma **probabilidade entre 0 e 1** por meio de uma **função específica** (*sigmoide*, para binária, *softmax* para classes múltiplas).

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$







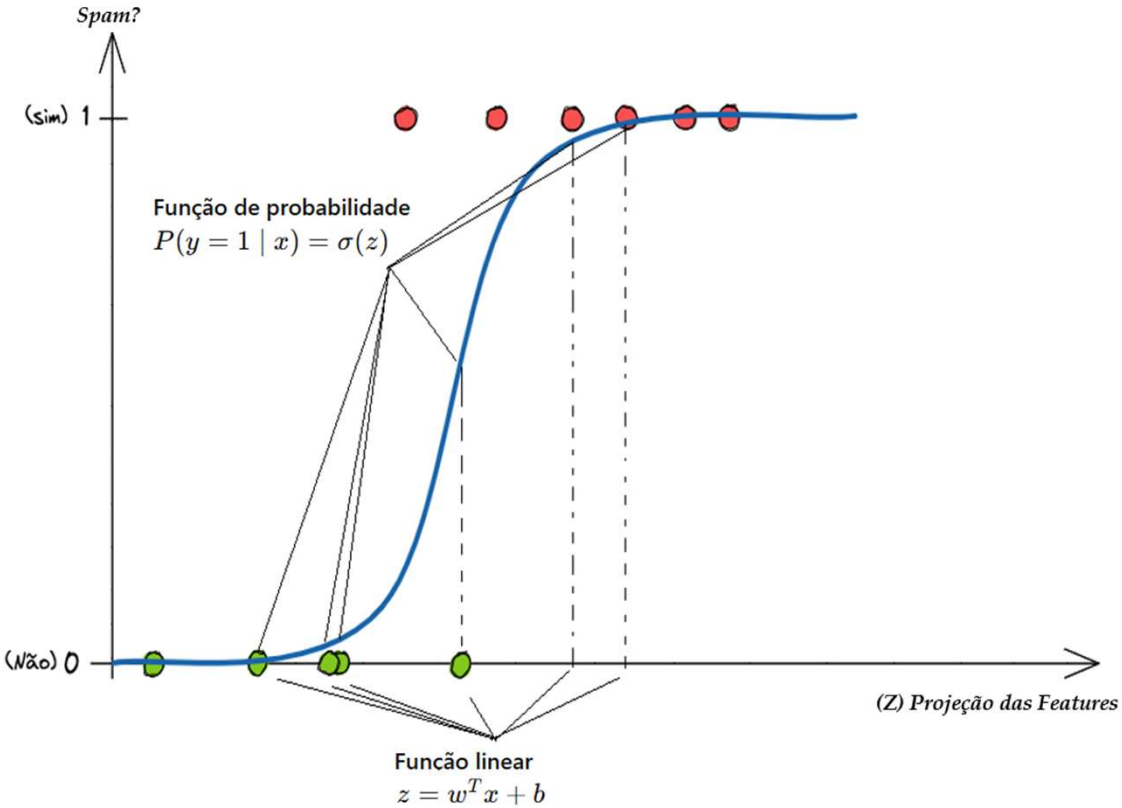
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

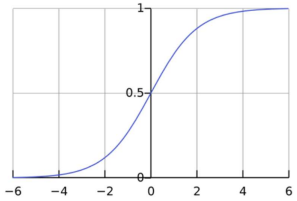
Regra de decisão

Os pontos no gráfico representam os **dados reais** do conjunto de treino, onde cada observação está **rotulada como 0 ou 1** (por exemplo, não spam ou spam). Ou seja, representam os **rótulos.**

Durante o treinamento, o modelo ajusta seus parâmetros (w, b) para gerar a "melhor" curva sigmoide representa a probabilidade de um registro **pertencer à classe 1.**

A curva **não separa perfeitamente os pontos.**

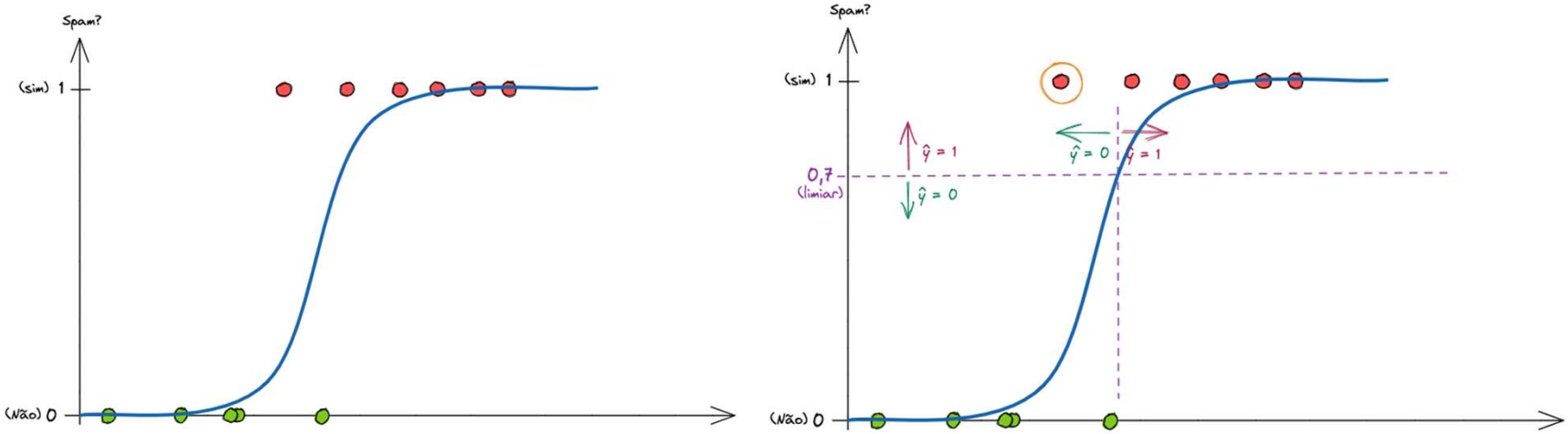




ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Regra de decisão

O modelo retorna uma probabilidade $p = \sigma(z)$ para a classe positiva. Para obter a classificação, define-se um limiar, geralmente 0.5.



Esse limiar não é obrigatório (pode ser ajustado conforme o custo do erro). Deixar passar fraude ou bloquear compra legítima? Posso definir limiar em 0.3, e ser conservador; ou 0.7, e ser agressivo.

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Quando usar?

Classificação de texto **linearmente separável** (aproximada)

Uso de **BoW** / **TF-IDF** (alta dimensionalidade esparsa)

Quando precisa de **baseline forte** e **interpretável**

Vantagens

Simples, rápido, estável

Funciona bem em alta dimensionalidade esparsa

Interpretável (pesos por palavra)

ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

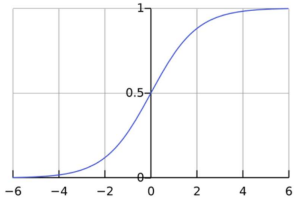
Quando evitar?

Relações não lineares complexas

Necessidade de capturar contexto/ordem (frases, semântica)

Quando *embeddings* trazem ganho claro

Mas eu poderia utilizar regressão logística com representações baseadas em *embeddings*? Sim, porém, por ser um modelo linear, ela não explora totalmente relações não lineares presentes em embeddings mais ricos (especialmente contextuais).



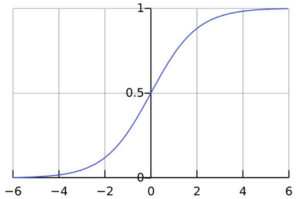
ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

A regressão logística clássica utiliza a função ***sigmoide*** (função logística) para problemas binários. Para problemas multiclasse, utiliza-se uma generalização desse modelo baseada na função ***softmax***

O modelo calcula inicialmente uma **combinação linear** das variáveis de entrada, cujo resultado pode variar de **−∞ a +∞**, e em seguida aplica uma função de transformação para convertê-lo em probabilidades. No caso binário, utiliza-se a função sigmoide; no caso multiclasse, a função softmax.

$$z = w \cdot X + b \quad \Rightarrow \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\mathrm{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

X representa o vetor de entrada (*features*) do modelo.

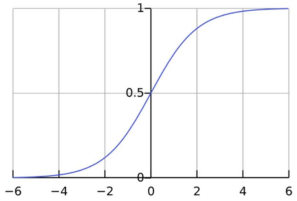
Passo (1) - Durante o treinamento, o modelo utiliza dados rotulados (X, y) para ajustar os parâmetros $\boldsymbol{w}$ (pesos) e $\boldsymbol{b}$ (viés). Com esses parâmetros, o modelo calcula a combinação linear:

$$z = w^T x + b$$

É esse valor z pode variar de $-\infty$ a $+\infty$.

Passo (2) – Calculado o valor de z, o próximo passo é calcular, via função sigmoide $\sigma(z)$, a probabilidade entre 0 e 1 utilizada na decisão de classe.

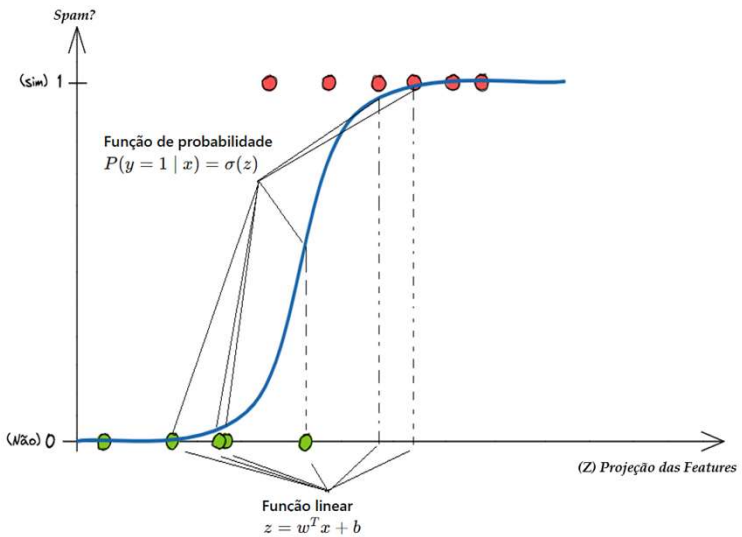
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

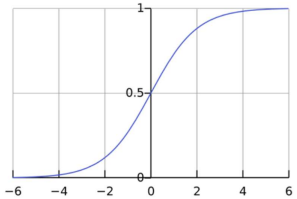


ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

Os parâmetros w(pesos) e b(viés) são **ajustados durante o treinamento** por meio de um processo de otimização que minimiza o erro entre as previsões do modelo e os valores reais. O detalhamento desse processo não será abordado neste momento.

$$(w, b) = \arg\min \sum_i \left[-y_i \log(\sigma(w^T x_i + b)) - (1 - y_i) \log(1 - \sigma(w^T x_i + b))\right]$$





ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

|V|: [excelente, ruim]

Suponha que o modelo já aprendeu:

- Peso (w) da palavra **"excelente" = +2**
- Peso (w) da palavra **"ruim" = −2**
- Viés $(b) = 0$

Pesos aprendidos e viés:

$$w = \begin{bmatrix} 2 \\ -2 \end{bmatrix} \quad b = 0$$

Para o texto "o produto é excelente":

$$X = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Cálculo para "o produto é excelente":

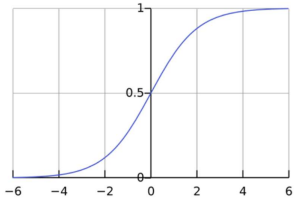
$$z = w^T X + b = \begin{bmatrix} 2 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0$$

$$z = w^T x + b$$

$$z = (2 \cdot 1) + (-2 \cdot 0) = 2$$

$$\sigma(2) = \frac{1}{1 + e^{-2}} \approx 0.88$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



ESPECIFICAÇÃO DE MODELOS SUPERVISIONADO - REGRESSÃO LOGÍSTICA

|V|: [excelente, ruim]

Suponha que o modelo já aprendeu:

- Peso (w) da palavra **"excelente" = +2**
- Peso (w) da palavra **"ruim" = −2**
- Viés $(b) = 0$

Pesos aprendidos e viés:

$$w = \begin{bmatrix} 2 \\ -2 \end{bmatrix} \quad b = 0$$

Para o texto "o produto é ruim":

$$X = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Cálculo para "o produto é ruim":

$$z = w^T X + b = \begin{bmatrix} 2 & -2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} + 0$$

$$z = w \cdot X + b$$

$$z = (2 \cdot 0) + (-2 \cdot 1) = -2$$

$$\sigma(-2) = \frac{1}{1 + e^2} \approx 0.12$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$